

FACULTAD DE INGENIERÍA

CARRERA DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

INFORME DE PROYECTO FINAL

Grupo 3-Autores:

Leon Sifuentes Kiara Xiomara (100%)

Loza Chanta Christian (100%)

Francisco Principe Yamil Zaid (100%)

Pérez Castro Jorge Abraham (100%)

Guerrero Guardamino Jeffry (100%)

Curso:

Base de Datos Avanzadas y Big Data

Docente:

MARLENE PILAR AGUILAR SUARES

Lima – Perú

2026-2

INDICES

	CONTENIDO
INDICES	2
RESUMEN	3
CAPÍTULO I: INTRODUCCIÓN	4
1.1. PROPUESTAS	4
1.1.1 Propuesta 01: Sistema integrado de base segura, automatizada para pollería Peppers.....	4
1.1.2 Propuesta 02: Aplicación móvil de pedidos y delivery.....	4
1.1.3 Propuesta 03: Sistema web de reservas y fidelizacion de clientes.....	4
1.2. IMPACTOS.....	4
1.3. IMPORTANCIA DEL PROYECTO	5
1.4. PROCESO DE RECOLECCIÓN DE DATOS	6
1.5. TIPO DE INVESTIGACIÓN.....	6
1.6. POBLACIÓN Y MUESTRA	6
1.7. TECNICAS E INSTRUMENTOS DE RECOLECCIÓN DE DATOS	6
CAPÍTULO II: MARCO TEÓRICO	8
2.2. BASES TEÓRICAS	8
2.3. ANÁLISIS DEL PROBLEMA	8
2.3.1. IDENTIFICACIÓN Y FORMULACION DEL PROBLEMA	8
2.3.2. DEFINICIÓN DE OBJETIVOS.....	9
2.3.3. ALCANCE DE LA SOLUCIÓN.....	9
CAPITULO III: HERRAMIENTAS DE INGENIERÍA	10
3.1) Comparación de metodologías de trabajo y estándares.....	10
3.2) Selección de las herramientas hardware y software, fundamentado	10
CAPITULO IV: GENERACIÓN DE SOLUCIONES.....	10
4.1. ALTERNATIVAS DE SOLUCIÓN.....	10

4.2. CRONOGRAMA DEL PROYECTO	11
4.3. ANÁLISIS DE TECNOLOGÍAS	11
4.4. MODELADO DEL NEGOCIO.....	11
CAPÍTULO V: METODOLOGÍA DE DESARROLLO	13
5.1. AUTOMATIZACIÓN DE PROCESOS DE BASE DATOS	13
5.1.1. Definición de los requerimientos del negocio	13
5.1.2. Identificación de los stakeholders	14
5.1.3. Diccionario de Datos.....	14
5.1.4. Modelado de elementos de datos	18
5.1.5. Implementación de base de datos.....	20
5.2. ANÁLISIS DE RESULTADOS.....	22
5.3. DESPLIEGUE DE LA SOLUCIÓN	23
5.6. CONTRASTACIÓN DE REQUERIMIENTOS	23
CAPITULO VI: CONCLUSIONES Y RECOMENDACIONES.....	23
6.1. CONCLUSIONES	23
6.2. RECOMENDACIONES	24
CAPITULO VII: COMPROMISO ETICO EN EL EJERCICIO PROFESIONAL.....	24
7.1. COMPROMISO ETICO	24
7.2. CONSECUENCIAS ÉTICAS.....	24
REFERENCIAS BIBLIOGRÁFICAS.....	25

RESUMEN

El presente informe documenta el diseño e implementación de un sistema integrado de base de datos para la pollería Pepers, sede Los Olivos (Av. Santiago Antúnez de Mayolo 1319, Los Olivos, Lima). A partir de un diagnóstico realizado en el propio local, se identificaron problemas operativos de tiempos de espera prolongados, colas en caja, errores en la toma de pedidos, falta de coordinación entre caja y cocina y ausencia de trazabilidad de las operaciones. Para atender esta problemática se diseñó una solución que articula tres bloques:

- 1- automatización de la validación e ingesta de datos mediante procedimientos almacenados, triggers y control de excepciones en SQL Server
- 2- seguridad de la información mediante roles diferenciados, auditoría y políticas de respaldo
- 3- una capa de inteligencia de negocios con un Data modelado según la metodología de Ralph Kimball y un dashboard de indicadores clave.

El presente entregable corresponde a la rubrica dada durante el curso , quedando documentada la base para el desarrollo de negocios y big data.

Palabras clave: base de datos relacional, automatización, SQL Server, seguridad de datos, pollería Pepers.

ABSTRACT

Este informe documenta el diseño y la implementación de un sistema de base de datos integrado para la pollería Peppers, sede Los Olivos, en Lima, Perú. A partir de un diagnóstico realizado en el propio local, el equipo identificó problemas operativos como tiempos de espera prolongados, colas en la caja, errores en la toma de pedidos, falta de coordinación entre caja y cocina, y ausencia de trazabilidad de las operaciones. Para atender esta problemática, se diseñó una solución organizada en estos bloques:

- 1) automatización de la validación e ingesta de datos mediante procedimientos almacenados, triggers y manejo de excepciones en SQL Server
- 2) seguridad de la información mediante roles diferenciados, auditoría y políticas de respaldo

CAPÍTULO I: INTRODUCCIÓN

1.1. PROPUESTAS

Se evaluó tres propuestas orientadas a resolver la problemática operativa observada en la pollería Peppers, sede Los Olivos.

1.1.1 Propuesta 01: Sistema integrado de base segura, automatizada para pollería Peppers

- Descripción: Diseño de una base de datos relacional en SQL Server que estructura la información de pedidos, clientes, productos, empleados, turnos, pagos y comprobantes, e incorpora triggers que registran automáticamente los cambios de estado de los pedidos y la hora de cada pedido.
- Ventaja: permite reducir la falta de automatización y trazabilidad mejorandola.
- Desventajas: requiere mayor esfuerzo de modelado de datos y de desarrollo de scripts ; demanda coordinación estrecha con el encargado del local para validar reglas de negocio.

1.1.2 Propuesta 02: Aplicación móvil de pedidos y delivery

- Descripción : app móvil para que los clientes realicen pedidos y paguen desde su celular, con seguimiento del estado del pedido.
- Ventajas : mejora la experiencia del cliente que pide para llevar o delivery; reduce colas en caja para ese segmento de clientes.
- Desventajas : no resuelve los problemas de fondo en la coordinación interna entre caja, cocina y despacho; requiere una capa de backend de base de datos igualmente robusta

1.1.3 Propuesta 03: Sistema web de reservas y fidelización de clientes

- Descripción : **portal web para reservar mesas y acumular puntos de fidelización.**
- Ventajas: aporta valor comercial a mediano plazo y facilita la relación con clientes frecuentes.
- Desventajas: no atiende los problemas operativos urgentes detectados en el diagnóstico (tiempos de espera, errores de pedido, falta de trazabilidad)

Tras comparar las tres propuestas, la mejor opción que se ha elegido fue la Propuesta 01 por ser la que ataca directamente la causa raíz del problema diagnosticado reduciéndolo y mejorarlo.

1.2. IMPACTOS

Detallar el impacto que podría generar el proyecto en los diferentes contextos.

1.2.1 Impacto social

La reducción de tiempos de espera y errores en la atención mejora la experiencia de los comensales y del propio personal del local, disminuyendo el estrés operativo durante las horas de mayor afluencia.

1.2.2 Impacto cultural

El proyecto promueve, dentro de un negocio familiar tradicional del rubro de pollerías, la incorporación de una cultura de datos y de toma de decisiones basada en evidencia, en lugar de criterios puramente empíricos.

1.2.3 Impacto político

Si bien la pollería Peppers no es una entidad pública, la implementación de controles de seguridad y roles de acceso diferenciados sienta un precedente de buenas prácticas de protección de datos personales de clientes y colaboradores, alineado al espíritu de la Ley N.º 29733

1.2.4 Impacto ambiental

Un mejor control de inventario y stock de insumos (registrado en la entidad Producto) contribuye a reducir el desperdicio de alimentos al anticipar quiebres de stock y ajustar las compras a proveedores.

1.2.5 Impacto ético

El manejo responsable de los datos de clientes (nombre, dirección, documento de identidad) y de los datos laborales del personal exige políticas claras de acceso, auditoría y confidencialidad, evitando el uso indebido de la información.

1.2.6 Impacto económico

La automatización de la toma de pedidos y del control de caja reduce reprocesos, pérdidas por errores de pedido y tiempos muertos, lo que se traduce en un mejor aprovechamiento del personal y en una mayor rotación de mesas en horas punta.

1.3. IMPORTANCIA DEL PROYECTO

El proyecto convierte un problema operativo cotidiano siendo este la lentitud y el desorden en la atención de la pollería Pepers, un caso de aplicación real donde el equipo trabajó sobre un negocio real. Asimismo el proyecto demuestra cómo negocios

pequeños y medianos, que habitualmente no cuentan con sistemas automatizados, pueden beneficiarse de una arquitectura de datos ordenada, segura y consistente

1.4. PROCESO DE RECOLECCIÓN DE DATOS

- ✓ observación directa del flujo de atención al cliente, desde la recepción del pedido hasta la entrega del producto, cubriendo las áreas de caja, cocina y despacho
- ✓ entrevistas semiestructuradas al encargado del local y a un grupo de clientes y registro de evidencia fotográfica de los puntos críticos del proceso

1.5. TIPO DE INVESTIGACIÓN

La investigación es de tipo aplicada, con un enfoque mixto: cualitativo en la fase de diagnóstico (observación y entrevistas) y cuantitativo en la fase de medición de indicadores operativos (tiempos de atención, número de quejas, frecuencia de quiebres de stock) que sustentan el diseño de la solución. Es, además, de alcance descriptivo, pues busca caracterizar la situación actual del proceso de atención antes de diseñar la solución tecnológica.

1.6. POBLACIÓN Y MUESTRA

La población está conformada por el personal que labora en la pollería Pepers, sede Los Olivos (encargado, cajeros, cocineros y personal de despacho), y por los clientes que asisten al local durante las horas de mayor afluencia ,viernes, sábado y domingo, entre las 12:00 y las 15:00 horas y entre las 19:00 y las 22:00 horas con una muestra no probabilística por conveniencia conformada por el encargado del local, 2 cajeros, 2 cocineros y 15 clientes entrevistados durante dos jornadas de observación, seleccionados por su disponibilidad y disposición a participar durante la visita de campo.

1.7. TECNICAS E INSTRUMENTOS DE RECOLECCIÓN DE DATOS

Técnica	Instrumento	Validez / Confiabilidad
Observación directa	Guía de observación del flujo de atención (caja, cocina, despacho)	Validada por contraste entre los tres observadores del equipo en dos visitas distintas
Entrevista semiestructurada	Guion de preguntas al encargado del local	Aplicada por el mismo entrevistador en ambas

		visitas para mantener consistencia
Encuesta breve	Cuestionario de 5 preguntas cerradas a clientes sobre tiempo de espera y satisfacción	Piloteada con 3 clientes antes de su aplicación definitiva
Registro fotográfico	Ficha de evidencia fotográfica por área (caja, cola, despacho)	Fotografías con fecha y hora, contrastadas con los horarios de observación

CAPÍTULO II: MARCO TEÓRICO

2.2. BASES TEÓRICAS

Modelo Entidad-Relación y bases de datos relacionales

El modelo Entidad-Relación (E-R) permite representar de forma conceptual las entidades del negocio (Cliente, Pedido, Producto, Empleado, entre otras), sus atributos y las relaciones entre ellas, sirviendo de base para derivar el modelo relacional o físico que se implementa en el gestor de base de datos (SQL Server).

Transact-SQL: procedimientos almacenados, triggers y transacciones

SQL es el lenguaje procedural de SQL Server que permite encapsular lógica de negocio en procedimientos almacenados y funciones, reaccionar automáticamente a eventos DML mediante triggers, y garantizar la atomicidad de las operaciones mediante el control de transacciones

2.3. ANÁLISIS DEL PROBLEMA

2.3.1. IDENTIFICACIÓN Y FORMULACION DEL PROBLEMA

El presente diagnóstico se realizó en la pollería Peppers, sede Los Olivos, ubicada en la Av. Santiago Antúnez de Mayolo 1319, distrito de Los Olivos, Lima. El estudio tiene como propósito identificar y analizar los principales problemas operativos que afectan la atención al cliente en el establecimiento, con miras a proponer mejoras en su proceso de servicio. La empresa analizada es una pollería dedicada a la venta de pollo a la brasa y platos complementarios para consumo en el local y para llevar.

Las entrevistas realizadas al encargado y a algunos clientes permitieron confirmar que estos inconvenientes impactan en la satisfacción del usuario y en la eficiencia del proceso de atención. Asimismo, se recopiló evidencia fotográfica para sustentar las observaciones efectuadas durante el diagnóstico.

Tabla de problemas de calidad detectados durante las dos jornadas :

Problema	Registros afectados	Efecto en el diseño
Tiempos de espera > 20 min en horas punta	18 de 45 pedidos (40 %)	Trigger de auditoría de estados y KPI de hora pico
Errores en la toma de pedidos (plato o cantidad)	6 de 45 pedidos (13 %)	Validación en procedimiento (productos existentes, cantidades > 0, stock)
Pedidos sin priorización ni registro formal	Recurrente según el encargado	RegistroAuditoriaPedido y orden por Hora de registro
Colas > 8 personas en caja	2 de 2 jornadas	Registro de pagos automatizado y comprobante por pago

2.3.2. DEFINICIÓN DE OBJETIVOS

Objetivo general

Diseñar e implementar una base de datos relacional en SQL Server para la pollería Peppers, que respete el modelo Entidad-Relación conceptual del equipo e incorpore triggers de auditoría y de registro de demanda horaria, para dar trazabilidad a la atención de pedidos.

Objetivos específicos

Relevar en el local los problemas operativos y los requerimientos de información.

- Identificar los problemas operativos y los requerimientos de información.
- Diseñar el modelo Entidad-Relación y el modelo físico de 16 tablas.
- Implementar triggers para registrar cambios de estado y demanda horaria.
- Incorporar procedimientos almacenados, una función y transacciones.
- Aplicar controles de seguridad, respaldo y auditoría.
- Evaluar el uso complementario de MongoDB, Data Warehouse, Power BI y Spark.

2.3.3. ALCANCE DE LA SOLUCIÓN

- ✓ **Incluye:** base de datos relacional de 16 tablas en SQL Server para una sola sede; automatización con procedimientos, funciones y triggers; seguridad por roles, enmascaramiento, respaldo y auditoría de estados; módulo NoSQL opcional; Data Warehouse dimensional con ETL, dashboard de KPIs y prueba de escalabilidad con Spark sobre datos simulados.
- ✓ **No incluye (no soporta):** aplicación de caja móvil (Propuesta 02), reservas y fidelización (Propuesta 03), facturación electrónica con SUNAT, cifrado en reposo ni auditoría de lecturas, ni conclusiones sobre la demanda real del local, porque los datos de prueba son simulados.
- ✓ **Hipótesis.** El estudio es aplicado y de alcance descriptivo-tecnológico: busca construir y verificar una solución, no contrastar una relación causal entre variables; por eso no formula hipótesis estadística. Se plantean en su lugar criterios de éxito verificables: cada cambio de estado queda registrado (3 filas en la prueba de auditoría), no se puede entregar un pedido sin pago completo, no se permite stock negativo y la base restaurada coincide con la original.

CAPITULO III: HERRAMIENTAS DE INGENIERÍA

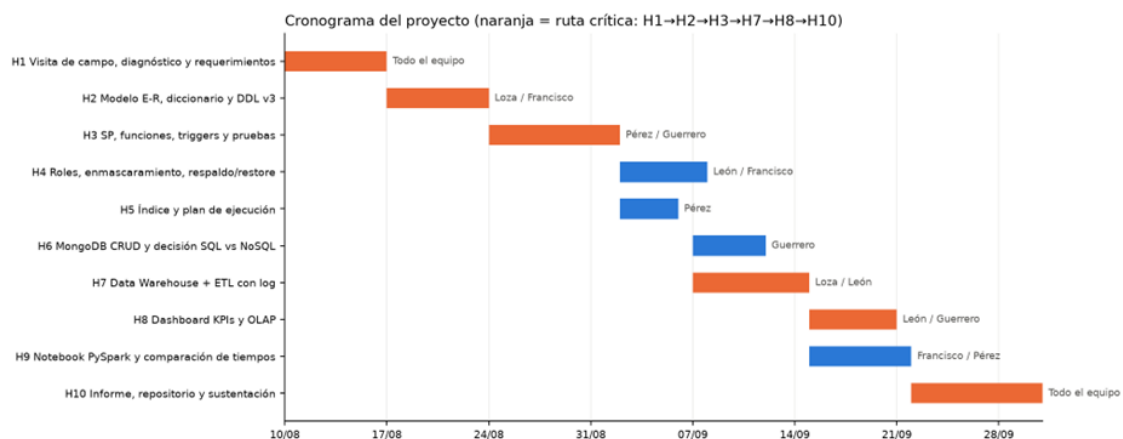
3.1) Comparación de metodologías de trabajo y estándares

Criterio	Scrum	Kanban	RUP
Adaptabilidad	Alta	Alta	Baja
Curva de aprendizaje	Media	Baja	Alta
Entregas parciales	Alta	Media	Media
Documentación	Ligera	Ligera	Extensa

3.2) Selección de las herramientas hardware y software, fundamentado

Herramienta	Función	Justificación
SQL Server 2022 Developer Edition	Motor relacional	Permite trabajar con T-SQL, triggers y transacciones.
SQL Server Management Studio	Administración y ejecución	Facilita la creación y prueba de scripts.
Git / GitHub	Control de versiones	Permite organizar archivos e historial.
Laptop mínimo 8 GB RAM y SSD	Entorno de desarrollo	Permite ejecutar SQL Server y SSMS.

3.3) cronograma



CAPITULO IV: GENERACIÓN DE SOLUCIONES

4.1. ALTERNATIVAS DE SOLUCIÓN

Criterio	Propuesta 01: BD relacional con triggers	Propuesta 02: App móvil de pedidos	Propuesta 03: Web de reservas / fidelización
Ataca la causa raíz del problema diagnosticado	Sí	Parcial	No
Cobertura de los contenidos del curso (modelado, T-SQL, triggers)	Completa	Parcial	Insuficiente
Complejidad técnica	Alta	Alta	Media
Tiempo estimado de desarrollo (6 semanas)	Factible	Ajustado	Factible
Valor percibido por el negocio a corto plazo	Alto	Medio	Medio

4.2. CRONOGRAMA DEL PROYECTO

Semana	Hito	Responsables
1	Visita, diagnóstico y requerimientos	Todo el equipo
2	Modelo E-R y diccionario	Loza Chanta, Christian
3	Base de datos y DDL	Francisco Principe, Yamil
4	Triggers y pruebas	Pérez Castro, Jorge
5	Automatización, seguridad y NoSQL	Todo el equipo
6	DW, BI, Spark y documentación	Todo el equipo

4.3. ANÁLISIS DE TECNOLOGÍAS

Categoría	Alternativas	Elección	Justificación
-----------	--------------	----------	---------------

SGBD relacional	SQL Server / MySQL / PostgreSQL	SQL Server	Se adapta al uso de T-SQL, triggers y transacciones.
NoSQL	MongoDB / otros	MongoDB	Permite documentos semiestructurados y CRUD.
BI	Power BI / hojas de cálculo	Power BI	Facilita indicadores y análisis interactivo.
Procesamiento distribuido	Spark / SQL tradicional	Spark para prueba comparativa	Permite evaluar procesamiento distribuido.

4.4. MODELADO DEL NEGOCIO

El proceso de atención al cliente en la pollería Peppers se organiza en cuatro etapas secuenciales:

1. Atención / toma de pedido: el cliente es recibido y su pedido se registra (mostrador o mesa), asociándolo a un cliente y a un tipo de entrega (local o para llevar).
2. Cocina: el pedido se traslada a cocina en el orden en que fue registrado; se prepara cada producto del detalle de pedido.
3. Caja: se genera el pago del pedido (efectivo, tarjeta o billetera digital) y el comprobante correspondiente (boleta o factura).
4. Despacho: el pedido preparado y pagado se entrega al cliente, actualizando su estado a "Entregado".

Reglas de negocio principales:

- ✓ Todo pedido debe estar asociado a un cliente (registrado o "cliente de mostrador" genérico) y a un empleado que lo atendió.
- ✓ Un pedido no puede marcarse como "Entregado" si no cuenta con un pago registrado por un monto igual o mayor al total del detalle de pedido.
- ✓ El stock de un producto se descuenta al confirmarse el detalle de pedido y se valida que no sea negativo.

CAPÍTULO V: METODOLOGÍA DE DESARROLLO

5.1. AUTOMATIZACIÓN DE PROCESOS DE BASE DATOS

5.1.1. Definición de los requerimientos del negocio

Requerimientos funcionales (RF):

Código	Requerimiento	Script	ESTADO
RF01	Registrar clientes con nombre, dirección y documento de identidad	Tabla Cliente; Pedido.ID_Cliente es FK NOT NULL, por lo que no se puede registrar un pedido sin cliente válido	completado
RF02	Registrar pedidos con su detalle (productos, cantidades y subtotales)	Tablas Pedido y DetallePedido; el subtotal se almacena pero no se calcula automáticamente	parcial
RF03	Controlar el stock de productos al confirmarse cada pedido	Producto.Stock existe , sin validación ni descuento automático	No abordado
RF04	Registrar pagos y comprobantes asociados a cada pedido	Tablas Pago, Comprobante , MetodoPago, TipoComprobante; sin validar monto contra total	parcial
RF05	Registrar turnos y asistencia del personal	Tablas Turno y RegistroTurno; sin validación de pedidos fuera de turno	parcial
RF06	Dejar trazabilidad de cambios sobre pedidos y pagos	Trigger trg_Auditoria_EstadoPedido y tabla RegistroAuditoriaPedido	completado
RF07	Gestionar productos y proveedores	Tablas Producto y Proveedor; Producto.ID_Proveedor	completado
RF08	Registrar la hora de cada pedido para identificar la hora pico	Trigger trg_RegistrarDemandaHoraria y tabla RegistroDemandaHoraria	completado

Requerimientos no funcionales (RNF):

- RNF01 – Integridad referencial mediante claves y restricciones.
- RNF02 – Trazabilidad de cambios de estado con usuario y fecha-hora.
- RNF03 – Tolerancia a fallas mediante TRY/CATCH.
- RNF04 – Seguridad mediante roles y permisos diferenciados.

5.1.2. Identificación de los stakeholders

Stakeholder	Rol	Impacto
Propietario / gerencia	Aprueba la solución	Alto
Encargado	Valida reglas y necesidades	Alto
Cajeros	Registran pedidos y pagos	Alto
Cocineros y despacho	Preparan y entregan pedidos	Medio

5.1.3. Diccionario de Datos

A continuación se documentan las tablas del modelo relacional

Tabla	Campo	Tipo	Restricciones
TipoEntrega	ID_TipoEntrega	TINYINT	IDENTITY PRIMARY KEY
TipoEntrega	Nombre	VARCHAR(20)	NOT NULL
Cargo	ID_Cargo	TINYINT	IDENTITY PRIMARY KEY
Cargo	Nombre	VARCHAR(50)	NOT NULL
ConceptoPago	ID_ConceptoPago	INT	IDENTITY PRIMARY KEY
ConceptoPago	Nombre	VARCHAR(50)	NOT NULL
MetodoPago	ID_MetodoPago	TINYINT	IDENTITY PRIMARY KEY
MetodoPago	Nombre	VARCHAR(20)	NOT NULL
TipoComprobante	ID_TipoComprobante	INT	IDENTITY PRIMARY KEY
TipoComprobante	Nombre	VARCHAR(30)	NOT NULL
Cliente	ID_Cliente	INT	IDENTITY PRIMARY KEY
Cliente	Nombre	VARCHAR(100)	NOT NULL
Cliente	Direccion	VARCHAR(150)	NULL

Cliente	DNI_RUC	VARCHAR(20)	NOT NULL
Proveedor	ID_Proveedor	INT	IDENTITY PRIMARY KEY
Proveedor	Nombre	VARCHAR(100)	NOT NULL
Proveedor	Contacto	VARCHAR(100)	NULL
Proveedor	Telefono	VARCHAR(20)	NULL
Empleado	ID_Empleado	INT	IDENTITY PRIMARY KEY
Empleado	NumeroDocumento	VARCHAR(20)	NOT NULL
Empleado	Nombre	VARCHAR(50)	NOT NULL
Empleado	ApellidoPaterno	VARCHAR(50)	NOT NULL
Empleado	ApellidoMaterno	VARCHAR(50)	NULL
Empleado	Telefono	VARCHAR(20)	NULL
Empleado	ID_Cargo	TINYINT	NOT NULL; FK → Cargo(ID_Cargo)
Producto	ID_Producto	INT	IDENTITY PRIMARY KEY
Producto	Nombre	VARCHAR(100)	NOT NULL
Producto	Precio	DECIMAL(8,2)	NOT NULL
Producto	Categoria	VARCHAR(50)	NOT NULL
Producto	Stock	INT	NOT NULL DEFAULT 0
Producto	ID_Proveedor	INT	NOT NULL; FK → Proveedor(ID_Proveedor)
Turno	ID_Turno	SMALLINT	IDENTITY PRIMARY KEY
Turno	NombreTurno	VARCHAR(30)	NOT NULL
Turno	HoraInicio	TIME	NOT NULL
Turno	HoraFin	TIME	NOT NULL
Turno	ID_Empleado	INT	NOT NULL; FK → Empleado(ID_Empleado)
Pedido	ID_Pedido	INT	IDENTITY PRIMARY KEY

Pedido	Fecha	DATE	NOT NULL DEFAULT GETDATE()
Pedido	Hora	TIME	NOT NULL DEFAULT GETDATE()
Pedido	Estado	VARCHAR(20)	NOT NULL DEFAULT 'Pendiente'
Pedido	ID_Cliente	INT	NOT NULL
Pedido	ID_Empleado	INT	NOT NULL
Pedido	ID_TipoEntrega	TINYINT	NOT NULL; FK → TipoEntrega(ID_TipoEntrega)
RegistroTurno	ID_Registro	INT	IDENTITY PRIMARY KEY
RegistroTurno	Estado	CHAR(1)	NOT NULL
RegistroTurno	Fecha	DATE	NOT NULL DEFAULT GETDATE()
RegistroTurno	HoraEntrada	DATETIMEOF FSET	NULL
RegistroTurno	HoraSalida	DATETIMEOF FSET	NULL
RegistroTurno	ID_Empleado	INT	NOT NULL; FK → Empleado(ID_Empleado)
RegistroTurno	ID_Turno	SMALLINT	NOT NULL
DetallePedido	ID_Detalle	INT	IDENTITY PRIMARY KEY
DetallePedido	Cantidad	INT	NOT NULL
DetallePedido	PrecioUnitario Hist	DECIMAL(8,2)	NOT NULL
DetallePedido	Subtotal	DECIMAL(8,2)	NOT NULL
DetallePedido	ID_Pedido	INT	NOT NULL
DetallePedido	ID_Producto	INT	NOT NULL; FK → Producto(ID_Producto)
Pago	ID_Pago	INT	IDENTITY PRIMARY KEY
Pago	Monto	DECIMAL(8,2)	NOT NULL
Pago	Fecha	DATE	NOT NULL DEFAULT GETDATE()
Pago	ID_Pedido	INT	NOT NULL
Pago	ID_MetodoPago	TINYINT	NOT NULL; FK → MetodoPago(ID_MetodoPago)

Comprobante	ID_Comprobante	INT	IDENTITY PRIMARY KEY
Comprobante	Fecha	DATE	NOT NULL DEFAULT GETDATE()
Comprobante	ID_TipoComprobante	INT	NOT NULL; FK → TipoComprobante(ID_TipoComprobante)
Comprobante	ID_Pago	INT	NOT NULL UNIQUE
PagoEmpleado	ID_PagoEmpleado	INT	IDENTITY PRIMARY KEY
PagoEmpleado	Monto	DECIMAL(8,2)	NOT NULL
PagoEmpleado	Fecha	DATE	NOT NULL DEFAULT GETDATE()
PagoEmpleado	ID_ConceptoPago	INT	NOT NULL; FK → ConceptoPago(ID_ConceptoPago)
PagoEmpleado	ID_Empleado	INT	NOT NULL
RegistroAuditoria Pedido	ID_Log	INT	IDENTITY PRIMARY KEY
RegistroAuditoria Pedido	ID_Pedido	INT	NOT NULL
RegistroAuditoria Pedido	EstadoAnterior	VARCHAR(20)	NOT NULL
RegistroAuditoria Pedido	EstadoNuevo	VARCHAR(20)	NOT NULL
RegistroAuditoria Pedido	UsuarioSQL	VARCHAR(100)	NOT NULL
RegistroAuditoria Pedido	FechaHoraCambio	DATETIME2	NOT NULL DEFAULT SYSDATETIME()
RegistroDemanda Horaria	ID_Registro	INT	IDENTITY PRIMARY KEY
RegistroDemanda Horaria	ID_Pedido	INT	NOT NULL; FK → Pedido(ID_Pedido)
RegistroDemanda Horaria	Fecha	DATE	NOT NULL
RegistroDemanda Horaria	HoraPedido	TIME	NOT NULL
RegistroDemanda Horaria	HoraEntera	TINYINT	NOT NULL

5.1.4. Modelado de elementos de datos

El modelo de datos se desarrolló en dos niveles: un modelo conceptual Entidad-Relación siendo FIGURA 1 que representa las entidades del negocio y sus relaciones, y un modelo relacional/físico, derivado del anterior, en el que cada entidad se convierte en una tabla con sus columnas, tipos de dato, llave primaria (PK) y llaves foráneas (FK). El modelo físico está documentado por el script DDL v2 en anexos

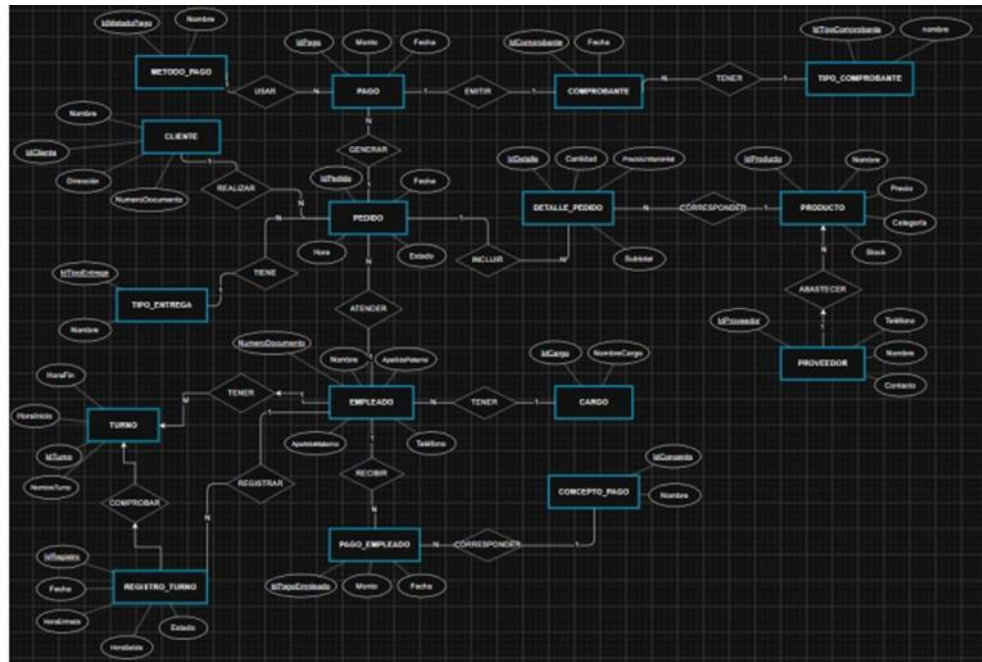


FIGURA 1 . Modelo relacional / físico de la base de datos

Este segundo diagrama corresponde al modelo relacional (o físico) obtenido a partir del modelo Entidad-Relación anterior. Aquí cada entidad se convierte en una tabla con sus columnas, tipos de dato (NUMBER, VARCHAR2, DATE, etc.), llave primaria (PK) y llaves foráneas (FK) que materializan las relaciones identificadas previamente. Este modelo se utiliza para generar el script de creación de la base de datos (DDL), ya que define cómo se almacenarán, vincularán y consultarán los datos del sistema de la pollera Peppers en el gestor de base de datos.

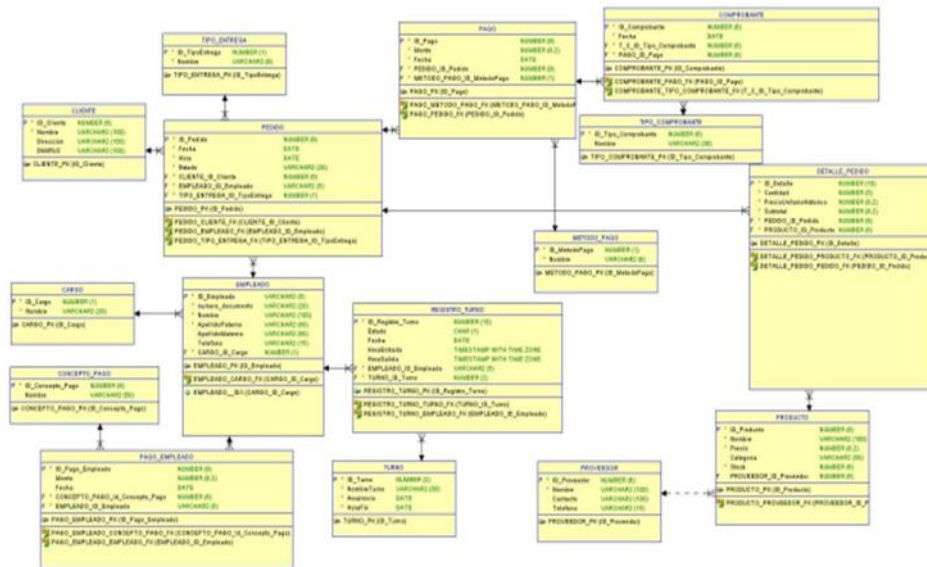


Figura 2. Modelo relacional / físico de la base de datos

Este segundo diagrama corresponde al modelo relacional (o físico) obtenido a partir del modelo Entidad-Relación anterior. Aquí cada entidad se convierte en una tabla con sus columnas, tipos de dato (NUMBER, VARCHAR2, DATE, etc.), llave primaria (PK) y llaves foráneas (FK) que materializan las relaciones identificadas previamente. Este modelo se utiliza para generar el script de creación de la base de datos (DDL), ya que define cómo se almacenarán, vincularán y consultarán los datos del sistema de la pollería Peppers en el gestor de base de datos.

Relaciones del modelo físico (una fila por llave foránea del DDL v2):

Tabla (lado N)	Columna FK	Tabla referenciada (lado 1)	Cardinalidad
Empleado	ID_Cargo	Cargo(ID_Cargo)	N:1
Producto	ID_Proveedor	Proveedor(ID_Proveedor)	N:1
Turno	ID_Empleado	Empleado(ID_Empleado)	N:1
Pedido	ID_TipoEntrega	TipoEntrega(ID_TipoEntrega)	N:1
RegistroTurno	ID_Empleado	Empleado(ID_Empleado)	N:1
DetallePedido	ID_Producto	Producto(ID_Producto)	N:1
Pago	ID_MetodoPago	MetodoPago(ID_MetodoPago)	N:1
Comprobante	ID_TipoComprobante	TipoComprobante(ID_TipoComprobante)	N:1
PagoEmpleado	ID_ConceptoPago	ConceptoPago(ID_ConceptoPago)	N:1

5.1.5. Implementación de base de datos

Los scripts de esta sección son los entregados y probados por el equipo: ddl_v2_corregido_polleria_peppers.sql y triggers_polleria_peppers.sql que se muestra en los anexos

a) **Script DDL v2.** Este crea las 16 tablas en tres niveles según sus dependencias:

1. tablas sin dependencias: TipoEntrega, Cargo, ConceptoPago, MetodoPago, TipoComprobante, Cliente, Proveedor

2. tablas que dependen de una entidad: Empleado, Producto, Turno

3. tablas que dependen de varias entidades Pedido, RegistroTurno, DetallePedido, Pago, Comprobante, PagoEmpleado

El cambio clave respecto a la versión anterior es que Turno tiene una FK a Empleado, para respetar la cardinalidad Turno (M) – Empleado 1 del diagrama conceptual. El script incluye al inicio los DROP TABLE en orden inverso a las dependencias, de modo que puede volver a ejecutarse.

B) Triggers: El script de triggers crea las tablas de log y los dos triggers DML sobre Pedido:

Estados de pedidos:

trg_Auditoria_EstadoPedido : registra en RegistroAuditoriaPedido cada cambio del campo Estado de un pedido, guardando el pedido, el estado anterior, el estado nuevo, el usuario SQL y la fecha-hora. Se activa solo con UPDATE y únicamente cuando la columna Estado forma parte del cambio (IF UPDATE(Estado)); además, el WHERE i.Estado <> d.Estado evita registrar actualizaciones que no cambian el estado.

```
CREATE OR ALTER TRIGGER trg_Auditoria_EstadoPedido
ON Pedido
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    IF UPDATE(Estado)
    BEGIN
        BEGIN TRY
            INSERT INTO RegistroAuditoriaPedido (ID_Pedido, EstadoAnterior,
            EstadoNuevo, UsuarioSQL)
            SELECT
                i.ID_Pedido,
                d.Estado,
                i.Estado,
                SUSER_SNAME()
            FROM inserted i
            INNER JOIN deleted d ON i.ID_Pedido = d.ID_Pedido
            WHERE i.Estado <> d.Estado;
        END TRY
        BEGIN CATCH
            PRINT 'Advertencia: no se pudo registrar auditoría. Error: ' +
            ERROR_MESSAGE();
        END CATCH
    END
END
```

END

END

triggers_pol...19\UPN (70)   ddl_v2_corre...C19\UPN (69))

```

64
65      */
66
67      CREATE OR ALTER TRIGGER trg_Auditoria_EstadoPedido
68      ON Pedido
69      AFTER UPDATE
70      AS
71      BEGIN
72          SET NOCOUNT ON;
73
74          IF UPDATE(Estado)
75          BEGIN TRY
76              INSERT INTO RegistroAuditoriaPedido (ID_Pedido, EstadoAnterior, EstadoNuevo, UsuarioSQL)
77              SELECT
78                  i.ID_Pedido,
79                  d.Estado,
80                  i.Estado,
81                  i.Estado,
82                  SUSER_SNAME()
83              FROM inserted i
84              INNER JOIN deleted d ON i.ID_Pedido = d.ID_Pedido
85              WHERE i.Estado <> d.Estado;
86          END TRY
87          BEGIN CATCH
88              PRINT 'Advertencia: no se pudo registrar auditoría. Error: ' + ERROR_MESSAGE();
89          END CATCH
90      END

```

90 %  No se encontraron problemas.

 Resultados  Mensajes

	ID_Log	ID_Pedido	EstadoAnterior	EstadoNuevo	UsuarioSQL	FechaHoraCambio
1	1	1	Entregado	En preparación	ULNLABD703PC19\UPN	2026-10-06 18:33:53.4310485
2	2	1	En preparación	Listo	ULNLABD703PC19\UPN	2026-10-06 18:33:53.4319661
3	3	1	Listo	Entregado	ULNLABD703PC19\UPN	2026-10-06 18:33:53.4319661

b) Demanda de hora/registro

trg_RegistrarDemandaHoraria : después de insertar un pedido, registra en la tabla RegistroDemandaHoraria la fecha, la hora exacta y la hora entera (0-23) del pedido, para poder identificar la hora de mayor demanda del local. Ante un error informa una advertencia sin bloquear el registro del pedido.

```

CREATE OR ALTER TRIGGER trg_RegistrarDemandaHoraria
ON Pedido
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        INSERT INTO RegistroDemandaHoraria (ID_Pedido, Fecha, HoraPedido, HoraEntera)
        SELECT
            i.ID_Pedido,
            i.Fecha,

```

```

        i.Hora,
        DATEPART(HOUR, i.Hora) -- extrae solo la hora (0-23)
    FROM inserted i;
END TRY
BEGIN CATCH
    PRINT 'Advertencia: no se pudo registrar la demanda horaria. Error: ' +
    ERROR_MESSAGE();
END CATCH
END

```

```

97
98 */
99
100 CREATE OR ALTER TRIGGER trg_RegistrarDemandaHoraria
101 ON Pedido
102 AFTER INSERT
103 AS
104 BEGIN
105     SET NOCOUNT ON;
106
107     BEGIN TRY
108         INSERT INTO RegistroDemandaHoraria (ID_Pedido, Fecha, HoraPedido, HoraEntera)
109         SELECT
110             i.ID_Pedido,
111             i.Fecha,
112             i.Hora,
113             DATEPART(HOUR, i.Hora) -- extrae solo la hora (0-23)
114         FROM inserted i;
115     END TRY
116     BEGIN CATCH
117         PRINT 'Advertencia: no se pudo registrar la demanda horaria. Error: ' + ERROR_MESSAGE();
118     END CATCH
119 END
120 GO

```

90 % No se encontraron problemas.

Resultados

Mensajes

	ID_Log	ID_Pedido	EstadoAnterior	EstadoNuevo	UsuarioSQL	FechaHoraCambio
1	1	1	Entregado	En preparación	ULNLABD703PC19\UPN	2026-10-06 18:33:53.4310485
2	2	1	En preparación	Listo	ULNLABD703PC19\UPN	2026-10-06 18:33:53.4319661
3	3	1	Listo	Entregado	ULNLABD703PC19\UPN	2026-10-06 18:33:53.4319661

	ID_Registro	ID_Pedido	Fecha	HoraPedido	HoraEntera
26	26	67	2026-10-06	12:42:00.0000000	12
27	27	68	2026-10-06	20:16:00.0000000	20
28	28	69	2026-10-06	20:26:00.0000000	20
29	29	70	2026-10-06	20:22:00.0000000	20
30	30	71	2026-10-06	21:37:00.0000000	21
31	31	72	2026-10-06	12:22:00.0000000	12
32	32	73	2026-10-06	12:34:00.0000000	12
33	33	74	2026-10-06	12:51:00.0000000	12

c) Datos de prueba. El script inserta dos empleados y ejecuta dos pruebas: la **Prueba 1** crea un pedido y lo cambia de estado tres veces, tras lo cual RegistroAuditoriaPedido debe mostrar tres filas de trazabilidad; la **Prueba 2** simula 40 pedidos con una distribución horaria ponderada (55% entre 19:00 y 22:00, 25% entre 12:00 y 14:00 y 20% entre 09:00 y 19:00) y consulta RegistroDemandaHoraria agrupando por HoraEntera para obtener la hora pico.

Justificación de diseño. Se optó por registrar la auditoría y la demanda mediante triggers, y no en la capa de aplicación, porque el trigger captura el cambio sin importar desde qué aplicación o usuario se realice, y centraliza la trazabilidad en la base de datos sin depender de que cada aplicación cliente implemente su propio registro.

5.1.6 Objetivos de automatización

Objeto	Tipo	Propósito / parámetros	Comportamiento ante excepciones
sp_RegistrarPedido	Procedimiento	Registra cabecera + detalle y descuento stock. @ID_Cliente, @ID_Empleado, @ID_TipoEntrega, @Detalle (TVP), @ValidarTurno, @ID_Pedido OUT	Validaciones 50001–50006 (THROW); transacción única: ante cualquier error, ROLLBACK total, registro en LogEventos y re-lanza el error
sp_RegistrarPago	Procedimiento	Registra pago parcial/total y comprobante. @ID_Pedido, @Monto, @ID_MetodoPago, @ID_TipoComprobante	Bloqueo UPDLOCK del pedido; monto > saldo → 50013 y ROLLBACK; si falla el comprobante: ROLLBACK parcial al SAVEPOINT (se conserva el pago, queda WARN)
sp_CambiarEstadoPedido	Procedimiento	Permite solo Pendiente → En preparación → Listo → Entregado	50021 si la transición no es válida; propaga el error del trigger de entrega
sp_LogEvento	Procedimiento	Escribe en LogEventos (nivel, origen, número, mensaje)	Se invoca después del ROLLBACK para que el log persista
trg_Auditoria_EstadoPedido	Trigger DML (auditoría)	Registra estado anterior/nuevo, usuario y fecha-hora	TRY/CATCH: un fallo del log no bloquea la operación (RNF03)
trg_RegistrarDemandaHoraria	Trigger DML	Registra hora exacta y hora entera de cada pedido	TRY/CATCH con advertencia
trg_Pedido_ValidarEntrega	Trigger (integridad)	Impide Entregado sin detalle o con saldo > 0	ROLLBACK + LogEventos + error 50030
trg_Detalle_ValidarIntegridad	Trigger (integridad)	Subtotal = Cantidad × Precio; bloquea cambios a pedidos entregados	ROLLBACK + LogEventos + errores 50031/50032
fn_TotalPedido, fn_SaldoPedido, fn_EmpleadoEnTurno	Funciones escalares	Total, saldo y verificación de turno (incluye turnos que cruzan medianoche)	Sin excepciones; devuelven 0 / 0 / BIT
fn_DemandaPorHora	Función con valor de tabla	Pedidos por hora y ranking en un rango de fechas	–

5.2. ANÁLISIS DE RESULTADOS

Comportamiento de los scripts y lo que permiten concluir:

- **Auditoría (Prueba 1).** El trigger trg_Auditoria_EstadoPedido registra cada cambio de estado de un pedido con su estado anterior, estado nuevo, usuario y fecha-hora, dando trazabilidad al proceso Pendiente → En preparación → Listo → Entregado. Esto responde al problema de falta de registro formal y de coordinación entre caja y

cocina identificado en el diagnóstico. [Insertar aquí captura del resultado de SELECT * FROM RegistroAuditoriaPedido.]

- **Demanda horaria (Prueba 2).** El trigger trg_RegistrarDemandaHoraria genera una fila en RegistroDemandaHoraria por cada pedido creado, y la consulta con GROUP BY HoraEntera entrega el conteo de pedidos por hora. [Insertar aquí captura del resultado.]

Limitaciones identificadas:

- Los datos de la Prueba 2 son simulados, con una distribución horaria definida por el propio script (55% noche, 25% almuerzo, 20% resto). Por eso el resultado de la hora pico refleja los parámetros de la simulación y **no** constituye evidencia de la demanda real; demuestra que el trigger captura y agrupa correctamente la hora. La hora pico real debe validarse con datos de varias semanas de operación antes de usarse para decidir la dotación de personal.
- El script de triggers usa versiones simplificadas de Pedido y Empleado (ver nota de implementación). Con el esquema completo del DDL v2, un INSERT en Pedido requiere además ID_Cliente e ID_TipoEntrega.
- La auditoría cubre solo el UPDATE del campo Estado en Pedido; no registra INSERT, DELETE ni cambios en Pago o Producto.

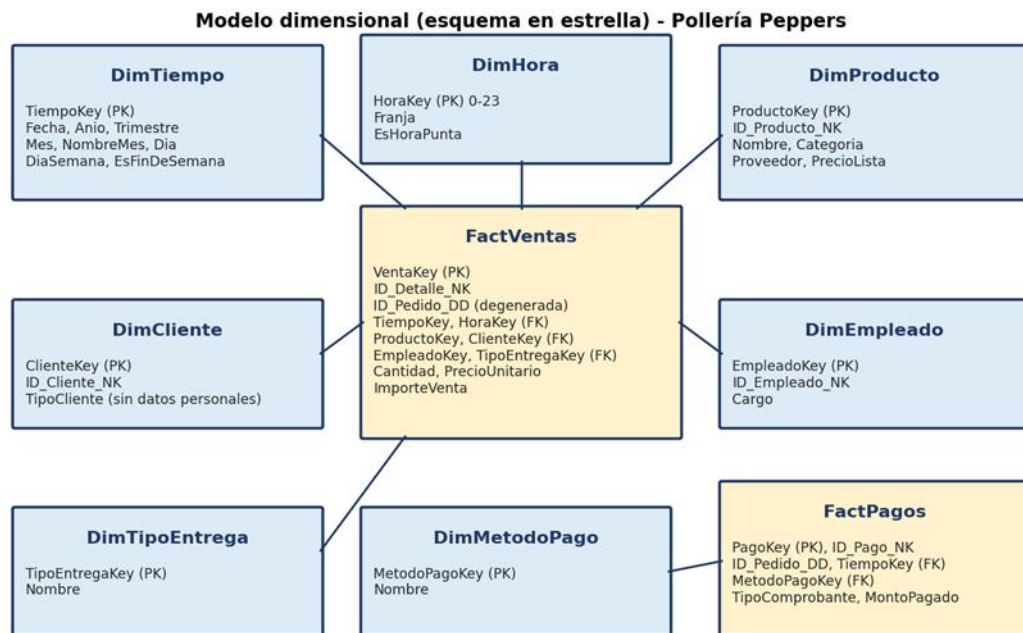
5.3. DESPLIEGUE DE LA SOLUCIÓN

Plan de despliegue:

1. Instalación de SQL Server 2022 Developer Edition en el equipo designado por el encargado del local.
2. Ejecución del script ddl_v2_corregido_polleria_peppers.sql para crear la estructura de base de datos.
3. Ejecución del script de triggers y de los procedimientos almacenados.
4. Creación de roles y usuarios (rol_administrador, rol_analista, rol_auditor) y asignación al personal real del local.
5. En una base de pruebas separada, ejecutar triggers_polleria_peppers.sql para crear las tablas de log, los triggers y los datos de prueba.
6. Verificar el resultado con SELECT * FROM RegistroAuditoriaPedido y con la consulta de hora pico sobre RegistroDemandaHoraria.

5.4 Data warehouse

Se diseñó un modelo dimensional según Ralph Kimball con dos tablas de hechos y siete dimensiones. **FactVentas** (grano: una línea de detalle de pedido entregado) y **FactPagos** (grano: un pago). Dimensiones: DimTiempo, DimHora, DimProducto, DimCliente, DimEmpleado, DimTipoEntrega y DimMetodoPago. **Privacidad por diseño:** DimCliente solo guarda el tipo de cliente; no se cargan nombre, DNI/RUC ni dirección.

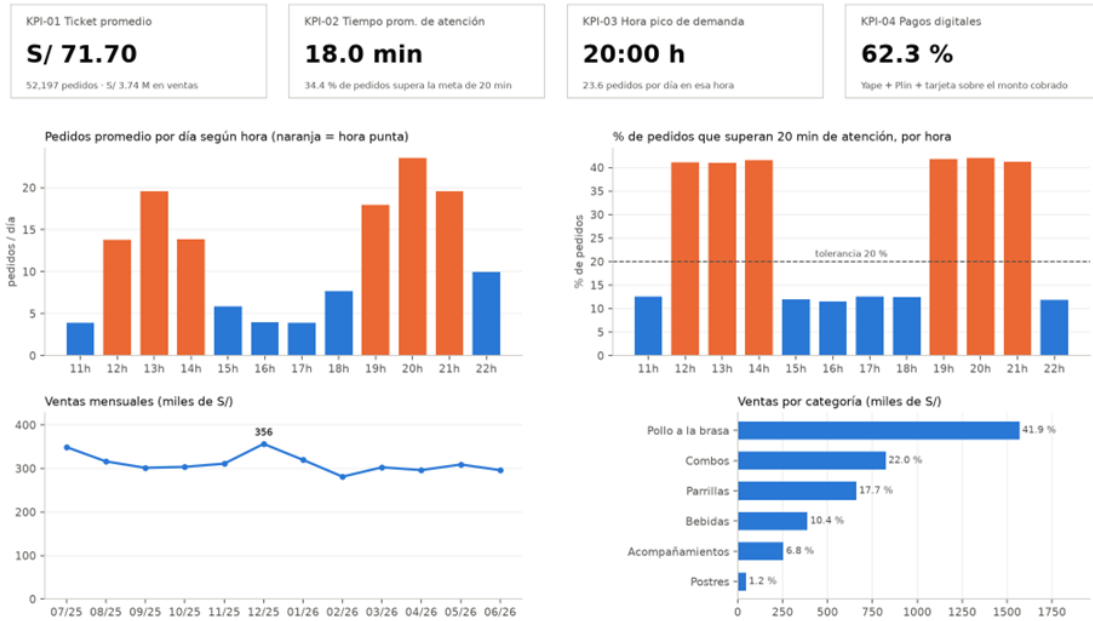


5.5 Dashboard BI , analisis y KPIs

En esta parte se representaría el día y horas presentada en la demanda y como afecta al tiempo de atención además de claro los productos que sostienen las ventas y cuales tienen baja rotación

Dashboard de operación - Pollería Peppers, sede Los Olivos (jul-2025 a jun-2026)

Fuente: Data Warehouse PeppersDW (FactVentas, FactAtencion, FactPagos). Datos simulados calibrados con observación de campo.



a) Metadatos y gobernanza de los KPI

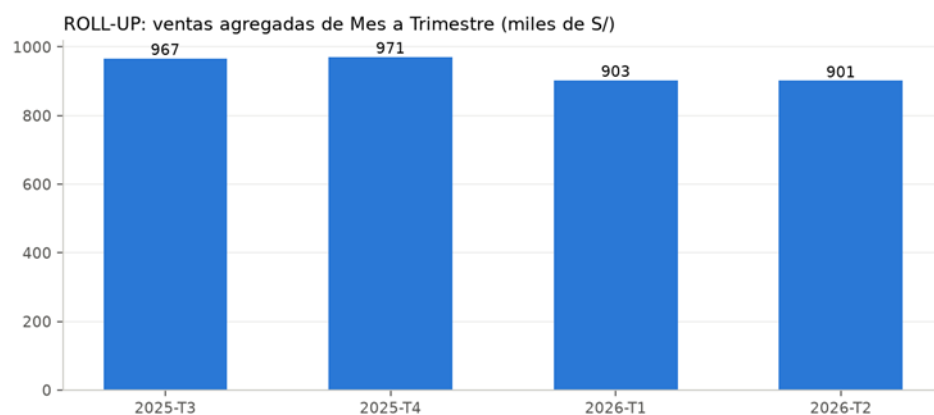
KPI	Fórmula	Fuente	Responsable / frecuencia	Criterio de calidad	Valor
KPI-01 Ticket promedio	$\text{SUM}(\text{ImporteVenta}) / \text{pedidos distintos}$	FactVentas	Encargado / diaria	Subtotales recalculados; 0 claves nulas	S/ 71,70
KPI-02 Tiempo de atención y % > 20 min	$\text{AVG}(\text{MinutosAtencion}); \text{pedidos con SuperaMeta}=1 / \text{pedidos con tiempo}$	FactAtencion	Encargado / diaria	Cobertura: 98 % de pedidos con hora de entrega	18,0 min; 34,4 %
KPI-03 Hora pico	Hora con más pedidos promedio por día	FactAtencion + DimHora	Encargado / semanal	Cada pedido tiene hora (completitud 100 %)	20:00 (23,6 ped/día)
KPI-04 Pagos digitales	$\text{Monto en Yape+Plin+Tarjeta} / \text{monto total}$	FactPagos + DimMetodoPago	Caja / mensual	Método estandarizado (8 222 corregidos)	62,3 %

Calidad y gobernanza :Completitud: 98,0 % de los pedidos tiene tiempo de atención y 100 % tiene hora de registro. Consistencia: 0 subtotales distintos de cantidad x precio después del ETL y 0 claves sustitutas nulas. Trazabilidad: cada KPI declara su tabla de hechos de origen y cada carga deja una fila por paso en ETL_Log. Seguridad analítica: el dashboard se construye sobre el DW sin datos personales, de modo que puede compartirse con el personal sin exponer clientes.

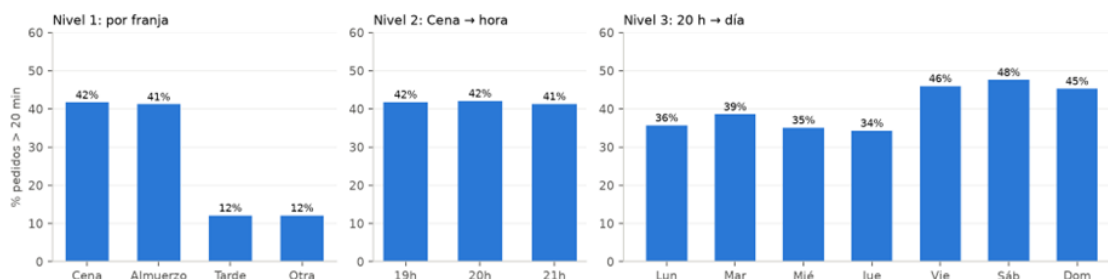
b) Operaciones OLAP

Roll-up (Mes → Trimestre → Año → Total) con GROUP BY ROLLUP: el tercer trimestre de 2025 sumó S/ 966 725, el cuarto S/ 971 079, el primero de 2026 S/ 903 356 y el segundo S/ 901 424, para un total anual de S/ 3 742 584. Julio y diciembre

destacan por las gratificaciones



DRILL-DOWN sobre FactAtencion: % de pedidos que superan la meta de 20 min



5.6. CONTRASTACIÓN DE REQUERIMIENTOS

Tabla donde muestre los indicadores establecidos por la empresa, frente a los requerimientos documentados.

Indicador de la empresa / necesidad	Requerimiento	Estado
Trazabilidad de cambios de pedido	RF06	COMPLETADO (solo cambios de Estado en Pedido)
Hora pico de demanda para planificación de personal	RF08	COMPLETADO a nivel de captura de datos (datos simulados)
Registro de pagos y comprobantes	RF04	PARCIAL (estructura de tablas; sin procedimiento ni validación)
Registro de turnos y asistencia	RF05	PARCIAL (estructura de tablas; sin procedimiento ni validación)
Registro de clientes, productos y proveedores	RF01, RF07	COMPLETADO a nivel de estructura
Control de stock sin valores negativos	RF03	NO ABORDADO
Pedido entregado solo si está pagado	Regla de negocio	NO ABORDADO

CAPITULO VI: CONCLUSIONES Y RECOMENDACIONES

6.1. CONCLUSIONES

- Se diseñó e implementó un modelo relacional de 16 tablas para la pollería Peppers que representa las cardinalidades del diagrama Entidad-Relación conceptual del equipo, incluyendo la corrección de la relación Turno–Empleado, cumpliendo el objetivo de contar con una base de datos estructurada y consistente.
- Se implementó un trigger de auditoría que deja trazabilidad automática de cada cambio de estado de un pedido, atendiendo la falta de registro formal identificada en el diagnóstico.
- Se implementó un trigger de registro de demanda horaria que, con datos simulados, permite obtener el conteo de pedidos por hora; su utilidad para decidir la dotación de personal depende de contar con datos reales.

6.2. RECOMENDACIONES

- Alinear el script de triggers con el esquema completo del DDL v2, de modo que los triggers se prueben sobre las tablas Pedido y Empleado definitivas.
- Implementar procedimientos almacenados para registrar pedidos y pagos, validando stock y monto pagado, y calculando subtotales y totales.
- Agregar restricciones CHECK (por ejemplo, sobre Estado y Cantidad) y ampliar la auditoría a Pago y Producto.

CAPITULO VII: COMPROMISO ETICO EN EL EJERCICIO PROFESIONAL

7.1. COMPROMISO ETICO

Como futuros ingenieros de sistemas, el equipo asume el compromiso de gestionar con **honestidad** la información real del negocio, sin alterar ni presentar datos ficticios como si fueran evidencia de campo; de actuar con **responsabilidad** técnica al diseñar controles de seguridad e integridad que protejan tanto la información del negocio como los datos personales de clientes y colaboradores; de mantener una postura de **solidaridad** con el encargado y el personal del local, priorizando soluciones que reduzcan su carga de trabajo en lugar de complejizarla; de actuar con **respeto** hacia el negocio familiar tradicional que es Peppers, validando cada decisión técnica con su encargado antes de imponerla; y de practicar la **inclusión social**, diseñando un sistema utilizable por personal con distintos niveles de familiaridad tecnológica (cajeros, cocineros), sin excluir a quienes tienen menor experiencia digital.

7.2. CONSECUENCIAS ÉTICAS

Como futuros ingenieros de sistemas, el equipo asume el compromiso de gestionar con **honestidad** la información real del negocio, sin alterar ni presentar datos ficticios como si fueran evidencia de campo; de actuar con **responsabilidad** técnica al diseñar controles de seguridad e integridad que

protejan tanto la información del negocio como los datos personales de clientes y colaboradores; de mantener una postura de **solidaridad** con el encargado y el personal del local, priorizando soluciones que reduzcan su carga de trabajo en lugar de complejizarla; de actuar con **respeto** hacia el negocio familiar tradicional que es Peppers, validando cada decisión técnica con su encargado antes de imponerla; y de practicar la **inclusión social**, diseñando un sistema utilizable por personal con distintos niveles de familiaridad tecnológica (cajeros, cocineros), sin excluir a quienes tienen menor experiencia digital.

REFERENCIAS BIBLIOGRÁFICAS

Macías, M., & Gómez, M. (2015). *Introducción a Apache Spark: para empezar a programar el Big Data*. Editorial UOC. <https://elibro.bibliotecaupn.elogim.com/es/lc/upnorte/titulos/58454>
<https://elibro.bibliotecaupn.elogim.com/es/lc/upnorte/titulos/230290>

ANEXOS

Evidencia fotografica:



Anexo 1: ddl_v2_corregido_polleria_peppers.sql

```
/* ===== PROYECTO: Sistema BD -  
Pollería Peppers (sede Los Olivos) CURSO: Base de Datos Avanzadas y Big Data - CIIN1021P DDL v2 -  
CORREGIDO para respetar EXACTAMENTE las cardinalidades del diagrama entidad-relación conceptual  
entregado por el equipo.
```

CAMBIO CLAVE respecto a la versión anterior:

- TURNO ahora tiene FK a EMPLEADO (ID_Empleado), porque el diagrama marca la relación TENER como TURNO(M) -- EMPLEADO(1): un empleado puede tener M turnos, y cada turno pertenece a un solo

empleado. Antes TURNO era una tabla independiente (catálogo compartido), lo cual no coincidía con el diagrama.

Todas las demás cardinalidades fueron verificadas contra el diagrama y se mantienen igual que en la versión anterior. ===== */

```
-- CREATE DATABASE PolleriaPeppers; -- GO -- USE PolleriaPeppers; -- GO
```

```
IF OBJECT_ID('Comprobante') IS NOT NULL DROP TABLE Comprobante; IF OBJECT_ID('Pago') IS NOT NULL DROP
TABLE Pago; IF OBJECT_ID('DetallePedido') IS NOT NULL DROP TABLE DetallePedido; IF
OBJECT_ID('RegistroTurno') IS NOT NULL DROP TABLE RegistroTurno; IF OBJECT_ID('Pedido') IS NOT NULL
DROP TABLE Pedido; IF OBJECT_ID('PagoEmpleado') IS NOT NULL DROP TABLE PagoEmpleado; IF
OBJECT_ID('Turno') IS NOT NULL DROP TABLE Turno; IF OBJECT_ID('Producto') IS NOT NULL DROP TABLE
Producto; IF OBJECT_ID('Empleado') IS NOT NULL DROP TABLE Empleado; IF OBJECT_ID('Cliente') IS NOT NULL
DROP TABLE Cliente; IF OBJECT_ID('Proveedor') IS NOT NULL DROP TABLE Proveedor; IF
OBJECT_ID('TipoEntrega') IS NOT NULL DROP TABLE TipoEntrega; IF OBJECT_ID('Cargo') IS NOT NULL DROP
TABLE Cargo; IF OBJECT_ID('ConceptoPago') IS NOT NULL DROP TABLE ConceptoPago; IF
OBJECT_ID('MetodoPago') IS NOT NULL DROP TABLE MetodoPago; IF OBJECT_ID('TipoComprobante') IS NOT NULL
DROP TABLE TipoComprobante; GO
```

```
/* ===== 1. TABLAS SIN DEPENDENCIAS ===== */
```

```
CREATE TABLE TipoEntrega ( ID_TipoEntrega TINYINT IDENTITY(1,1) PRIMARY KEY, Nombre VARCHAR(20) NOT
NULL -- 'Local', 'Para llevar' ); GO
```

```
CREATE TABLE Cargo ( ID_Cargo TINYINT IDENTITY(1,1) PRIMARY KEY, Nombre VARCHAR(50) NOT NULL --
'Cajero', 'Cocinero', 'Mozo' ); GO
```

```
CREATE TABLE ConceptoPago ( ID_ConceptoPago INT IDENTITY(1,1) PRIMARY KEY, Nombre VARCHAR(50) NOT NULL
-- 'Sueldo', 'Bono' ); GO
```

```
CREATE TABLE MetodoPago ( ID_MetodoPago TINYINT IDENTITY(1,1) PRIMARY KEY, Nombre VARCHAR(20) NOT NULL
-- 'Efectivo', 'Tarjeta', 'Yape' ); GO
```

```
CREATE TABLE TipoComprobante ( ID_TipoComprobante INT IDENTITY(1,1) PRIMARY KEY, Nombre VARCHAR(30) NOT
NULL -- 'Boleta', 'Factura' ); GO
```

```
CREATE TABLE Cliente ( ID_Cliente INT IDENTITY(1,1) PRIMARY KEY, Nombre VARCHAR(100) NOT NULL,
Direccion VARCHAR(150) NULL, DNI_RUC VARCHAR(20) NOT NULL ); GO
```

```
CREATE TABLE Proveedor ( ID_Proveedor INT IDENTITY(1,1) PRIMARY KEY, Nombre VARCHAR(100) NOT NULL,
Contacto VARCHAR(100) NULL, Telefono VARCHAR(20) NULL ); GO
```

```
/* ===== 2. TABLAS QUE DEPENDEN DE UNA ENTIDAD ===== */
```

```
-- EMPLEADO (N) --TENER-- (1) CARGO CREATE TABLE Empleado ( ID_Empleado INT IDENTITY(1,1) PRIMARY KEY,
NumeroDocumento VARCHAR(20) NOT NULL, Nombre VARCHAR(50) NOT NULL, ApellidoPaterno VARCHAR(50) NOT
NULL, ApellidoMaterno VARCHAR(50) NULL, Telefono VARCHAR(20) NULL, ID_Cargo TINYINT NOT NULL,
CONSTRAINT FK_Empleado_Cargo FOREIGN KEY (ID_Cargo) REFERENCES Cargo(ID_Cargo) ); GO
```

```
-- PRODUCTO (N) --ABASTECER-- (1) PROVEEDOR CREATE TABLE Producto ( ID_Producto INT IDENTITY(1,1)
PRIMARY KEY, Nombre VARCHAR(100) NOT NULL, Precio DECIMAL(8,2) NOT NULL, Categoria VARCHAR(50) NOT
NULL, Stock INT NOT NULL DEFAULT 0, ID_Proveedor INT NOT NULL, CONSTRAINT FK_Producto_Proveedor FOREIGN
KEY (ID_Proveedor) REFERENCES Proveedor(ID_Proveedor) ); GO
```

```
-- *** CORRECCIÓN *** TURNO (M) --TENER-- (1) EMPLEADO -- Cada turno pertenece a UN empleado; un
empleado puede tener VARIOS turnos. CREATE TABLE Turno ( ID_Turno SMALLINT IDENTITY(1,1) PRIMARY KEY,
NombreTurno VARCHAR(30) NOT NULL, -- ej. 'Mañana', 'Tarde', 'Noche' HoraInicio TIME NOT NULL, HoraFin
TIME NOT NULL, ID_Empleado INT NOT NULL, CONSTRAINT FK_Turno_Empleado FOREIGN KEY (ID_Empleado)
REFERENCES Empleado(ID_Empleado) ); GO
```

```
/* ===== 3. TABLAS QUE DEPENDEN DE VARIAS ENTIDADES ===== */
```

```
-- PEDIDO depende de CLIENTE (1:N), EMPLEADO (1:N vía ATENDER), TIPO_ENTREGA (1:N vía TIENE) CREATE
TABLE Pedido ( ID_Pedido INT IDENTITY(1,1) PRIMARY KEY, Fecha DATE NOT NULL DEFAULT CAST(GETDATE() AS
DATE), Hora TIME NOT NULL DEFAULT CAST(GETDATE() AS TIME), Estado VARCHAR(20) NOT NULL DEFAULT
'Pendiente', ID_Cliente INT NOT NULL, ID_Empleado INT NOT NULL, ID_TipoEntrega TINYINT NOT NULL,
CONSTRAINT FK_Pedido_Cliente FOREIGN KEY (ID_Cliente) REFERENCES Cliente(ID_Cliente), CONSTRAINT
```

```
FK_Pedido_Empleado FOREIGN KEY (ID_Empleado) REFERENCES Empleado(ID_Empleado), CONSTRAINT
FK_Pedido_TipoEntrega FOREIGN KEY (ID_TipoEntrega) REFERENCES TipoEntrega(ID_TipoEntrega) ); GO

-- REGISTRO_TURNO depende de EMPLEADO (1:N vía REGISTRAR) y TURNO (1:N vía COMPROBAR, -- cardinalidad
no rotulada explícitamente en el diagrama -> se asume 1:N convencional; -- confírmalo con la profesora
o agrégalo al diagrama para evitar ambigüedad) CREATE TABLE RegistroTurno ( ID_Registro INT
IDENTITY(1,1) PRIMARY KEY, Estado CHAR(1) NOT NULL, Fecha DATE NOT NULL DEFAULT CAST(GETDATE() AS
DATE), HoraEntrada DATETIMEOFFSET NULL, HoraSalida DATETIMEOFFSET NULL, ID_Empleado INT NOT NULL,
ID_Turno SMALLINT NOT NULL, CONSTRAINT FK_RegTurno_Empleado FOREIGN KEY (ID_Empleado) REFERENCES
Empleado(ID_Empleado), CONSTRAINT FK_RegTurno_Turno FOREIGN KEY (ID_Turno) REFERENCES Turno(ID_Turno)
); GO

-- DETALLE_PEDIDO (N) --INCLUIR-- (1) PEDIDO | (N) --CORRESPONDER-- (1) PRODUCTO CREATE TABLE
DetallePedido ( ID_Detalle INT IDENTITY(1,1) PRIMARY KEY, Cantidad INT NOT NULL, PrecioUnitarioHist
DECIMAL(8,2) NOT NULL, Subtotal DECIMAL(8,2) NOT NULL, ID_Pedido INT NOT NULL, ID_Producto INT NOT
NULL, CONSTRAINT FK_Detalle_Pedido FOREIGN KEY (ID_Pedido) REFERENCES Pedido(ID_Pedido), CONSTRAINT
FK_Detalle_Producto FOREIGN KEY (ID_Producto) REFERENCES Producto(ID_Producto) ); GO

-- PAGO (N) --GENERAR-- (1) PEDIDO (permite pagos parciales/divididos por pedido) -- PAGO (N) --USAR--
... espera, METODO_PAGO(1)--USAR--(N)PAGO CREATE TABLE Pago ( ID_Pago INT IDENTITY(1,1) PRIMARY KEY,
Monto DECIMAL(8,2) NOT NULL, Fecha DATE NOT NULL DEFAULT CAST(GETDATE() AS DATE), ID_Pedido INT NOT
NULL, ID_MetodoPago TINYINT NOT NULL, CONSTRAINT FK_Pago_Pedido FOREIGN KEY (ID_Pedido) REFERENCES
Pedido(ID_Pedido), CONSTRAINT FK_Pago_Metodo FOREIGN KEY (ID_MetodoPago) REFERENCES
MetodoPago(ID_MetodoPago) ); GO

-- COMPROBANTE (1) --EMITIR-- (1) PAGO | (N) --TENER-- (1) TIPO_COMPROBANTE CREATE TABLE Comprobante (
ID_Comprobante INT IDENTITY(1,1) PRIMARY KEY, Fecha DATE NOT NULL DEFAULT CAST(GETDATE() AS DATE),
ID_TipoComprobante INT NOT NULL, ID_Pago INT NOT NULL UNIQUE, -- UNIQUE fuerza la cardinalidad 1:1 con
PAGO CONSTRAINT FK_Comprobante_Tipo FOREIGN KEY (ID_TipoComprobante) REFERENCES
TipoComprobante(ID_TipoComprobante), CONSTRAINT FK_Comprobante_Pago FOREIGN KEY (ID_Pago) REFERENCES
Pago(ID_Pago) ); GO

-- PAGO_EMPLEADO (N) --RECIBIR-- (1) EMPLEADO | (N) --CORRESPONDER-- (1) CONCEPTO_PAGO CREATE TABLE
PagoEmpleado ( ID_PagoEmpleado INT IDENTITY(1,1) PRIMARY KEY, Monto DECIMAL(8,2) NOT NULL, Fecha DATE
NOT NULL DEFAULT CAST(GETDATE() AS DATE), ID_ConceptoPago INT NOT NULL, ID_Empleado INT NOT NULL,
CONSTRAINT FK_PagoEmp_Concepto FOREIGN KEY (ID_ConceptoPago) REFERENCES ConceptoPago(ID_ConceptoPago),
CONSTRAINT FK_PagoEmp_Empleado FOREIGN KEY (ID_Empleado) REFERENCES Empleado(ID_Empleado) ); GO
```

Anexo 2: triggers_polleria_peppers.sql

```
IF OBJECT_ID('Comprobante') IS NOT NULL DROP TABLE Comprobante; IF OBJECT_ID('Pago') IS NOT NULL DROP
TABLE Pago; IF OBJECT_ID('PagoEmpleado') IS NOT NULL DROP TABLE PagoEmpleado; IF
OBJECT_ID('RegistroTurno') IS NOT NULL DROP TABLE RegistroTurno; IF OBJECT_ID('Turno') IS NOT NULL DROP
TABLE Turno; IF OBJECT_ID('DetallePedido') IS NOT NULL DROP TABLE DetallePedido; IF
OBJECT_ID('Producto') IS NOT NULL DROP TABLE Producto; GO

/* =====

1.  TABLAS BASE ===== */

IF OBJECT_ID('Pedido') IS NOT NULL DROP TABLE Pedido; IF OBJECT_ID('Empleado') IS NOT NULL DROP TABLE
Empleado; IF OBJECT_ID('RegistroAuditoriaPedido') IS NOT NULL DROP TABLE RegistroAuditoriaPedido; IF
OBJECT_ID('RegistroDemandaHoraria') IS NOT NULL DROP TABLE RegistroDemandaHoraria; GO

CREATE TABLE Empleado ( ID_Empleado INT IDENTITY(1,1) PRIMARY KEY, NumeroDocumento VARCHAR(20) NOT
NULL, Nombre VARCHAR(50) NOT NULL, ApellidoPaterno VARCHAR(50) NOT NULL ); GO

CREATE TABLE Pedido ( ID_Pedido INT IDENTITY(1,1) PRIMARY KEY, Fecha DATE NOT NULL DEFAULT
CAST(GETDATE() AS DATE), Hora TIME NOT NULL DEFAULT CAST(GETDATE() AS TIME), Estado VARCHAR(20) NOT
NULL DEFAULT 'Pendiente', -- Estados esperados: Pendiente, En preparación, Listo, Entregado ID_Empleado
INT NOT NULL, CONSTRAINT FK_Pedido_Empleado FOREIGN KEY (ID_Empleado) REFERENCES Empleado(ID_Empleado)
); GO

-- Tabla de LOG: auditoría de cambios de estado del pedido CREATE TABLE RegistroAuditoriaPedido (
ID_Log INT IDENTITY(1,1) PRIMARY KEY, ID_Pedido INT NOT NULL, EstadoAnterior VARCHAR(20) NOT NULL,
EstadoNuevo VARCHAR(20) NOT NULL, UsuarioSQL VARCHAR(100) NOT NULL, FechaHoraCambio DATETIME2 NOT NULL
DEFAULT SYSDATETIME() ); GO
```

```
-- Tabla de LOG: hora de cada pedido, para detectar la hora pico CREATE TABLE RegistroDemandaHoraria (
ID_Registro INT IDENTITY(1,1) PRIMARY KEY, ID_Pedido INT NOT NULL, Fecha DATE NOT NULL, HoraPedido TIME
NOT NULL, -- hora exacta del pedido, ej. 20:47:22 HoraEntera TINYINT NOT NULL, -- solo la hora, 0-23,
para agrupar fácil CONSTRAINT FK_Demanda_Pedido FOREIGN KEY (ID_Pedido) REFERENCES Pedido(ID_Pedido) );
GO
```

```
/* ===== 2. TRIGGER DE AUDITORÍA -
trg_Auditoria_EstadoPedido
```

```
*/
```

```
CREATE OR ALTER TRIGGER trg_Auditoria_EstadoPedido ON Pedido AFTER UPDATE AS BEGIN SET NOCOUNT ON;
```

```
IF UPDATE(Estado)
```

```
BEGIN
```

```
    BEGIN TRY
```

```
        INSERT INTO RegistroAuditoriaPedido (ID_Pedido, EstadoAnterior, EstadoNuevo, UsuarioSQL)
```

```
        SELECT
```

```
            i.ID_Pedido,
```

```
            d.Estado,
```

```
            i.Estado,
```

```
            SUSER_SNAME()
```

```
        FROM inserted i
```

```
        INNER JOIN deleted d ON i.ID_Pedido = d.ID_Pedido
```

```
        WHERE i.Estado <> d.Estado;
```

```
    END TRY
```

```
    BEGIN CATCH
```

```
        PRINT 'Advertencia: no se pudo registrar auditoría. Error: ' + ERROR_MESSAGE();
```

```
    END CATCH
```

```
END
```

```
END GO
```

```
/* ===== 3. TRIGGER DE DEMANDA -
trg_RegistrarDemandaHoraria
```

```
*/
```

```
CREATE OR ALTER TRIGGER trg_RegistrarDemandaHoraria ON Pedido AFTER INSERT AS BEGIN SET NOCOUNT ON;
```

```
BEGIN TRY
```

```
    INSERT INTO RegistroDemandaHoraria (ID_Pedido, Fecha, HoraPedido, HoraEntera)
```

```
    SELECT
```

```
        i.ID_Pedido,
```

```
        i.Fecha,
```

```
        i.Hora,
```

```
        DATEPART(HOUR, i.Hora) -- extrae solo la hora (0-23)
```

```
    FROM inserted i;
```

```
END TRY
```

```
BEGIN CATCH
```

```
    PRINT 'Advertencia: no se pudo registrar la demanda horaria. Error: ' + ERROR_MESSAGE();
```

```
END CATCH
```

```
END GO
```

```
/* ===== 4. DATOS DE PRUEBA Y
EJECUCIÓN ===== */
```

```
INSERT INTO Empleado (NumeroDocumento, Nombre, ApellidoPaterno) VALUES ('70112233', 'Kiara', 'Leon'),
('70223344', 'Christian', 'Loza');
```

```
-- ---- PRUEBA 1: cambio de estado -> dispara el trigger de auditoría ---- INSERT INTO Pedido
(ID_Empleado) VALUES (1); -- Pedido #1 UPDATE Pedido SET Estado = 'En preparación' WHERE ID_Pedido = 1;
UPDATE Pedido SET Estado = 'Listo' WHERE ID_Pedido = 1; UPDATE Pedido SET Estado = 'Entregado' WHERE
ID_Pedido = 1;

SELECT * FROM RegistroAuditoriaPedido; -- debe mostrar 3 filas de trazabilidad

-- ---- PRUEBA 2: simula 40 pedidos en el día (pollería de alta demanda), -- con distribución horaria
realista: -- 55% en la noche (19:00 - 22:00) <- hora pico -- 25% en el almuerzo (12:00 - 14:00) -- 20%
dispersos en el resto del día (09:00 - 19:00) ---- DECLARE @i INT = 1; DECLARE @totalPedidos INT = 40;
DECLARE @empleadoId INT; DECLARE @hora TIME; DECLARE @franja FLOAT;

WHILE @i <= @totalPedidos BEGIN SET @franja = RAND(CHECKSUM(NEWID()));

IF @franja < 0.55
    SET @hora = DATEADD(MINUTE, ABS(CHECKSUM(NEWID())) % 180, '19:00'); -- noche: 19:00-22:00
ELSE IF @franja < 0.80
    SET @hora = DATEADD(MINUTE, ABS(CHECKSUM(NEWID())) % 120, '12:00'); -- almuerzo: 12:00-14:00
ELSE
    SET @hora = DATEADD(MINUTE, ABS(CHECKSUM(NEWID())) % 600, '09:00'); -- resto: 09:00-19:00

SET @empleadoId = (ABS(CHECKSUM(NEWID())) % 2) + 1; -- empleado 1 o 2

INSERT INTO Pedido (Hora, ID_Empleado) VALUES (@hora, @empleadoId);

SET @i += 1;

END

SELECT * FROM RegistroDemandaHoraria; -- 1 fila por cada pedido creado

-- ---- CONSULTA CLAVE: hora pico de mayor demanda ---- SELECT HoraEntera AS Hora, COUNT(*) AS
CantidadPedidos FROM RegistroDemandaHoraria GROUP BY HoraEntera ORDER BY CantidadPedidos DESC; -- La
primera fila de este resultado es la HORA PICO del local.
```

Anexo 3: Enlaces del proyecto. Repositorio: <https://github.com/brammldk/BD-Y-BIG-DATA-T.FINAL>

Presentación: <https://canva.link/ddrg8t3eve5kbu5>